Invokedynamic is one of those JVM features that sounds scary but is actually very elegant once you see it in action.
I'll explain it **from zero**, then show **real Java code**, **bytecode flow**, and **how the JVM executes it**.

---

1 What is invokedynamic?

invokedynamic is a **JVM bytecode instruction** introduced in **Java 7**.

 Its job is to **delay method binding until runtime**.

Simple definition (interview-friendly):

invokedynamic allows the JVM to decide *at runtime* which method implementation should be called, instead of fixing it at compile time.

---

2 Why was invokedynamic introduced?

Before Java 7, JVM had only:

Instruction	Used for
invokestatic	Static methods
invokevirtual	Virtual methods
invokeinterface	Interface methods
invokespecial	Constructors, private methods

❌ These are **statically typed & early-bound**
❌ Very inefficient for **dynamic languages** and **lambdas**

Java needed:

✔Runtime flexibility

✔Better performance for lambdas

✔Support for dynamic languages (Groovy, Scala, JRuby)

 Hence **invokedynamic** was born.

---

3 Where is invokedynamic used?

- ✅ **Lambda expressions**
- ✅ **Method references**
- ✅ **String concatenation (Java 9+)**
- ✅ Dynamic languages on JVM

---

4 Simple Java Example (Lambda)

```
interface Add {
   int addition(int a, int b);
}

public class Test {
   public static void main(String[] args) {
      Add add = (a, b) -> a + b;
      System.out.println(add.addition(10, 20));
   }
}
```

---

5 What Compiler Generates (Behind the Scenes)

□ Step 1: Lambda body becomes a private method

```
private static int lambda$main$0(int a, int b) {
   return a + b;
}
```

□ Step 2: invokedynamic in bytecode

If you run:

```
javap -c Test
```

You'll see something like:

```
0: invokedynamic #0,  0  // InvokeDynamic #0:addition:()LAdd;
```

□ This is the key instruction.

---

6 How invokedynamic Works (Step-by-Step)

 Step 1: First time execution

When JVM hits invokedynamic:

- It **does NOT know** which method to call yet
- It calls a **bootstrap method**

 For lambdas, the bootstrap method is:

LambdaMetafactory.metafactory()

 Step 2: Bootstrap method job

The bootstrap method:

1. Receives method signature
2. Links functional interface (Add)
3. Binds it to:
4. lambda$main$0(int, int)
5. Creates a **CallSite**

 Step 3: CallSite caching

- JVM **remembers** the resolved method
- Next time, no lookup cost
- Direct jump to target method

 This makes lambdas **very fast**.

---

7 Runtime Execution Flow

```
main()
 ↓
invokedynamic
 ↓
Bootstrap Method (only once)
 ↓
CallSite created
 ↓
lambda$main$0(10, 20)
 ↓
return 30
```

---

8 Key Difference vs Normal Method Calls

Feature	Normal Call	Invokedynamic
Binding time	Compile time	Runtime
Flexibility	Low	Very High
Performance	Good	Excellent (after bootstrap)
JVM optimization	Limited	Advanced
Used by	Java ≤7	Java 8+ Lambdas

---

9 Simple Analogy

Think of invokedynamic like:

"I don't know which function to call **right now**…
When I reach there, I'll decide — and then I'll remember it forever."

Interview One-Liner ★

"invokedynamic is a JVM instruction that enables runtime method resolution. In Java 8, it is used to implement lambda expressions by dynamically linking a functional interface to its implementation using a bootstrap method."

Bonus: Why Lambda Is Faster Than Anonymous Class?

Lambda	Anonymous Class
Uses invokedynamic	Uses class loading
No extra .class file	Extra class generated
Can be cached	New object each time
JVM-friendly	Less optimizable

If you want next:

- ✔invokedynamic **diagram**
- ✔javap output explained line-by-line
- ✔String concatenation internals (Java 9+)
- ✔Capturing vs non-capturing lambdas